REPORT OF ASSIGNMENT 1

Part A: Stack ADT

- **What is an ADT ?** :

  An Abstract Data Type (ADT) is a logical description of a data structure that specifies **what** operations can be performed (like push or pop) but not **how** they are implemented internally.

- **Efficiency**:
  Stack operations like push and pop are O(1) because they only involve adding or removing an element from the end of the list, which does not require shifting other elements.

Part B & D: Complexity Analysis

Algorithm Paradigm	**Time complexity**	**space complicity**	
Factorial Recursion	O(n)	O(n)	
Fibonacci (naïve) Recursion	O(2^n)	O(n)	
Fibonacci (Memo) Dynamic Pro.	O(n)	O(n)	
Tower of Hanoi Recursion	O(2^n)	O(n)	
Binary Search & Conquer	O(log^n)	O(log^n)	Divide

Note on Fibonacci: Naive Fibonacci is slow because it recalculates the same subproblems (e.g., fib(2)) thousands of times.

Binary Search Recurrence:T(n)=T(n/2)+O(1)

= O(logn)

Part E: Paradigm Reflection:

1. Why Binary Search is Divide & Conquer

- **Divide:** The algorithm "divides" the problem into smaller subproblems by finding the middle element of the current array. It compares the search key to this middle element.
- **Conquer**: Based on the comparison, the algorithm "conquers" the problem by narrowing the search to only one half of the array.

1. If the key is smaller than the middle, the entire right half is discarded.
2. If the key is larger, the entire left half is discarded
3. **Recurrence and Complexity**
4. The efficiency of this approach is represented by the recurrence relation T(n) = T(n/2) + O(1) This means each step reduces the problem size by half (n/2) and takes constant time (O(1)) to perform the middle comparison.

Case Analysis:

- **Best Case**: O(1) — The key is found at the very first middle index.
- **Average/Worst Case**: O(log n) — The algorithm must continue dividing the array until only one element remains

Example: Searching for a Word in a Physical Dictionary Instead of reading every single page from the beginning (Linear Search), you open the dictionary to the middle. You check if your word comes before or after that page. You "divide" the book into two halves, "conquer" the correct half by ignoring the other, and repeat the process until you find the word. This is exactly how your **Recursive Binary Search** functions

2. Fibonacci memoized is Dynamic Programming (what problem does memo solve?)

The Problem: Overlapping Subproblems

In the naive recursive version of Fibonacci (fib_naive), the algorithm recalculates the same values multiple times. For example, to calculate fib(5), the computer must calculate fib(4) and fib(3). However, calculating

fib(4) *also* requires calculating fib(3) again. As n grows, this creates an exponential number of repeated function calls, making the process extremely slow.

he Solution: Memoization

Memoization solves this by creating a **"memo"** (usually a dictionary or an array) to store the result of each subproblem the first time it is calculated

- **Check Before Calculating**: Before the function performs any math, it checks the "memo" to see if the answer for that specific n is already there.
- **Store After Calculating**: If the value is not in the memo, the function calculates it once and then saves it before returning the result.
- **Instant Retrieval**: The next time the function is called with that same n, it returns the saved value instantly in O(1) time instead of restarting the recursion.

Why it is "Dynamic Programming"

Memoization follows the core principle of Dynamic Programming (DP): **Top-Down optimization**. It breaks the main problem into subproblems and ensures each unique subproblem is solved exactly **once**.

Impact on Efficiency
As you will show in your output.txt, this change is massive:

Naive Fibonacci: Has a Time Complexity of O(2^n) because the calls double at almost every level.

- **Memoized Fibonacci**: Has a Time Complexity of only O(n) because it only performs one calculation for each number from 1 to n

Example: Using a GPS Navigation App (like Google Maps) When a GPS calculates the best route, it often breaks the trip into smaller segments (intersections). If it has already calculated the fastest way to get from "Point A" to a "Major Intersection B," it saves that result in memory. If you take a slight detour, it doesn't recalculate everything from scratch; it uses the "memoized" (stored) segment to find the new path

quickly. This mirrors how **Fibonacci with Memoization** avoids redundant calculations.

3. Tower of Hanoi uses recursion (explain base case and smaller subproblem idea)

1. The Smaller Subproblem Idea

The Tower of Hanoi is a classic example of a problem that is solved by reducing it into smaller, identical versions of itself. To move a stack of n disks from a Source peg to a Destination peg, the algorithm follows a three-step logic:

1. Move the Sub-stack: Move the top n-1 disks from the Source to the Auxiliary peg. This is the "smaller subproblem" because you are now solving the same puzzle but for a smaller stack.
2. **The Actual Move**: Move the single largest disk (disk n) directly from the **Source** to the **Destination**.
3. **Complete the Stack**: Move the n-1 disks from the **Auxiliary** peg back onto the **Destination** peg (on top of disk n)

2. The Base Case

The base case is the condition that stops the recursion from continuing forever.

- **The Condition**: In the Tower of Hanoi, the base case occurs when $n = 1
- **The Action**: When there is only one disk to move, no sub-stacks need to be shifted. The algorithm simply moves that one disk directly from the Source to the Destination and stops the recursive calls for that branch

3. Complexity Summary for your Report

- **Number of Moves**: For n disks, the total moves required is 2^n – 1.
- **Time Complexity**: O(2^n), because every time you add a disk, the number of required moves roughly doubles.

- **Space Complexity**: O(n), which represents the maximum depth of the "call stack" (the memory used to keep track of the active function calls).

Example: Making Change for a Customer If a cashier needs to give you 80 cents in change, they don't look at every possible combination of coins. They use a "greedy" approach: they pick the largest denomination possible first (e.g., a 50-cent coin), then the next largest (20-cent), then another (10-cent) until the total is reached. At each step, they make the choice that looks best *at that moment* to reach the goal quickly.